1. Problem Title & Link

Title: **LeetCode 259: 3Sum Smaller**

Link: https://leetcode.com/problems/3sum-smaller/

2. Problem Statement (Short Summary)

You are given an integer array nums and an integer target.

Your task is to **count the number of triplets** (i, j, k) such that:

i < j < k

nums[i] + nums[j] + nums[k] < target

Return the **total number of such triplets**.

The goal is not to list the triplets but **count how many exist**.

3. Examples (Input → Output)

Example 1

Input

nums = [-2,0,1,3]

target = 2

Output

2

Explanation

Valid triplets:

(-2,0,1) = -1 < 2

(-2,0,3) = 1 < 2

Total = **2**

Example 2

Input

nums = [1,2,3]

target = 6

Output

1

Explanation

Triplet:

1 + 2 + 3 = 6

But condition requires **< target**, so answer = **0**

Example 3

Input

nums = [-1,1,-1,-1]

target = -1

Output

1

4. Constraints

0 ≤ nums.length ≤ 3500

-100 ≤ nums[i] ≤ 100

-100 ≤ target ≤ 100

Important observations:

- n can be **3500**
- $O(n^3)$ brute force is too slow
- Need **$O(n^2)$** solution

5. Core Concept (Pattern / Topic)

Two Pointers + Sorting

Why?

Because:

- We need ordered traversal
- Sorting allows efficient pair search
- Two pointers reduces complexity

6. Thought Process (Step-by-Step Explanation)

Step 1 — Brute Force

Try all triplets:

```
for i
  for j
    for k
```

Check:

nums[i] + nums[j] + nums[k] < target

Time Complexity

$O(n^3)$

For n = 3500 this is impossible.

Step 2 — Optimization Idea

If the array is **sorted**, we can use **two pointers**.

Sort array first.

Then:

Fix first number i.

Now find pairs (j,k) where

nums[i] + nums[j] + nums[k] < target

Step 3 — Key Insight

If

nums[i] + nums[left] + nums[right] < target

Then **all elements between left and right are valid**.

Why?

Because array is sorted.

nums[left+1] ≤ nums[right]

nums[left+2] ≤ nums[right]

...

So we can add:

right - left

triplets instantly.

7. Visual / Intuition Diagram (ASCII)

Sorted array example

nums = [-2,0,1,3]

i = -2

```
      L     R
[-2, 0, 1, 3]
```

Check

-2 + 0 + 3 = 1 < 2

Valid.

Since sorted:

(-2,0,1)

(-2,0,3)

Count = R - L

2 - 1 = 1

Move left forward.

8. Pseudocode

```
sort array

count = 0

for i in range(n-2):

   left = i + 1
   right = n - 1

   while left < right:

      sum = nums[i] + nums[left] + nums[right]

      if sum < target:
         count += right - left
         left += 1
      else:
         right -= 1

return count
```

9. Code Implementation

Python

```
class Solution:
   def threeSumSmaller(self, nums, target):
      nums.sort()
      n = len(nums)
      count = 0

      for i in range(n - 2):
         left = i + 1
         right = n - 1

         while left < right:
            total = nums[i] + nums[left] + nums[right]
```

```
            if total < target:
                count += right - left
                left += 1
            else:
                right -= 1

        return count
```

Java

```
class Solution {
    public int threeSumSmaller(int[] nums, int target) {
        Arrays.sort(nums);
        int n = nums.length;
        int count = 0;

        for (int i = 0; i < n - 2; i++) {

            int left = i + 1;
            int right = n - 1;

            while (left < right) {

                int sum = nums[i] + nums[left] + nums[right];

                if (sum < target) {
                    count += right - left;
                    left++;
                } else {
                    right--;
                }
            }
        }

        return count;
    }
}
```

10. Time & Space Complexity

Time Complexity

Sorting → O(n log n)

Two pointer loop → O(n^2)

Overall:

O(n^2)

Space Complexity

O(1)

Sorting in-place.

11. Common Mistakes / Edge Cases

1. Forgetting to **sort array**
2. Using count++ instead of count += right - left
3. Wrong pointer movement
4. Not handling small arrays (n < 3)
5. Integer overflow in other languages

12. Detailed Dry Run

Example

nums = [-2,0,1,3]

target = 2

Sorted:

[-2,0,1,3]

i	left	right	sum	action	count
0	-2	1	3	1<2	count+= 2
0	0	1	-1	valid	move left

Final answer:

2

13. Common Use Cases (Real-Life / Interview)

This pattern is used in:

- Financial risk calculations
- Finding combinations under threshold
- Resource allocation problems

- Budget constraint optimization

14. Common Traps

- Moving wrong pointer
- Forgetting sorted property
- Counting duplicates incorrectly
- Using brute force accidentally

15. Builds To (Related Problems)

Two Pointer progression:

LC 167 Two Sum II

LC 15 3Sum

LC 259 3Sum Smaller

LC 16 3Sum Closest

LC 18 4Sum

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Brute Force	$O(n^3)$	Too slow
Two Pointers	$O(n^2)$	Optimal

17. Why This Solution Works

Sorting allows us to **monotonically adjust pointers**.

When a triplet satisfies the condition, **all elements between pointers also satisfy it**, enabling batch counting.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Find **triplets with sum ≤ target**
2. Find **actual triplets instead of count**
3. Extend to **4Sum smaller**
4. Solve using **streaming input**